ECE 622: Modeling & Verification of Embedded Systems

Spring 2026: Lab 2: Verification of Timed Automata using UPPAAL

Prof. Daniel Holcomb

Submitted by:	Jaideep Khare and Shu-Lin Shih
Student ID:	35193853 and 35199856
Lab No.:	2
Submission Date:	25th March, 2026

Overview

This report presents model construction, simulation, and symbolic verification for Lab 2 using UPPAAL. Part A analyzes timed automata for the Viking bridge-crossing problem under increasingly complex scenarios. Part B develops and verifies a timed traffic-light and pedestrian-control system, including safety analysis and a robustness extension required for ECE 622.

The workflow used in all sections was: (1) encode model changes in UPPAAL templates/system declarations, (2) state explicit verification queries, (3) run verifier checks, and (4) inspect symbolic or concrete traces to extract schedules and explain results.

1 Part A: Bridge-Crossing Timed Automata

A.1: Four Vikings within 60 minutes

Determine whether all four Vikings with crossing times 5 min, 10 min, 20 min, *and* 25 min *can reach the safe side within* 60 min, *and provide a valid schedule.*

Model checking query:

```
E<> Vik_1.safe && Vik_2.safe && Vik_3.safe && Vik_4.safe && time <= 60
```

The query is satisfiable, so a feasible execution exists.

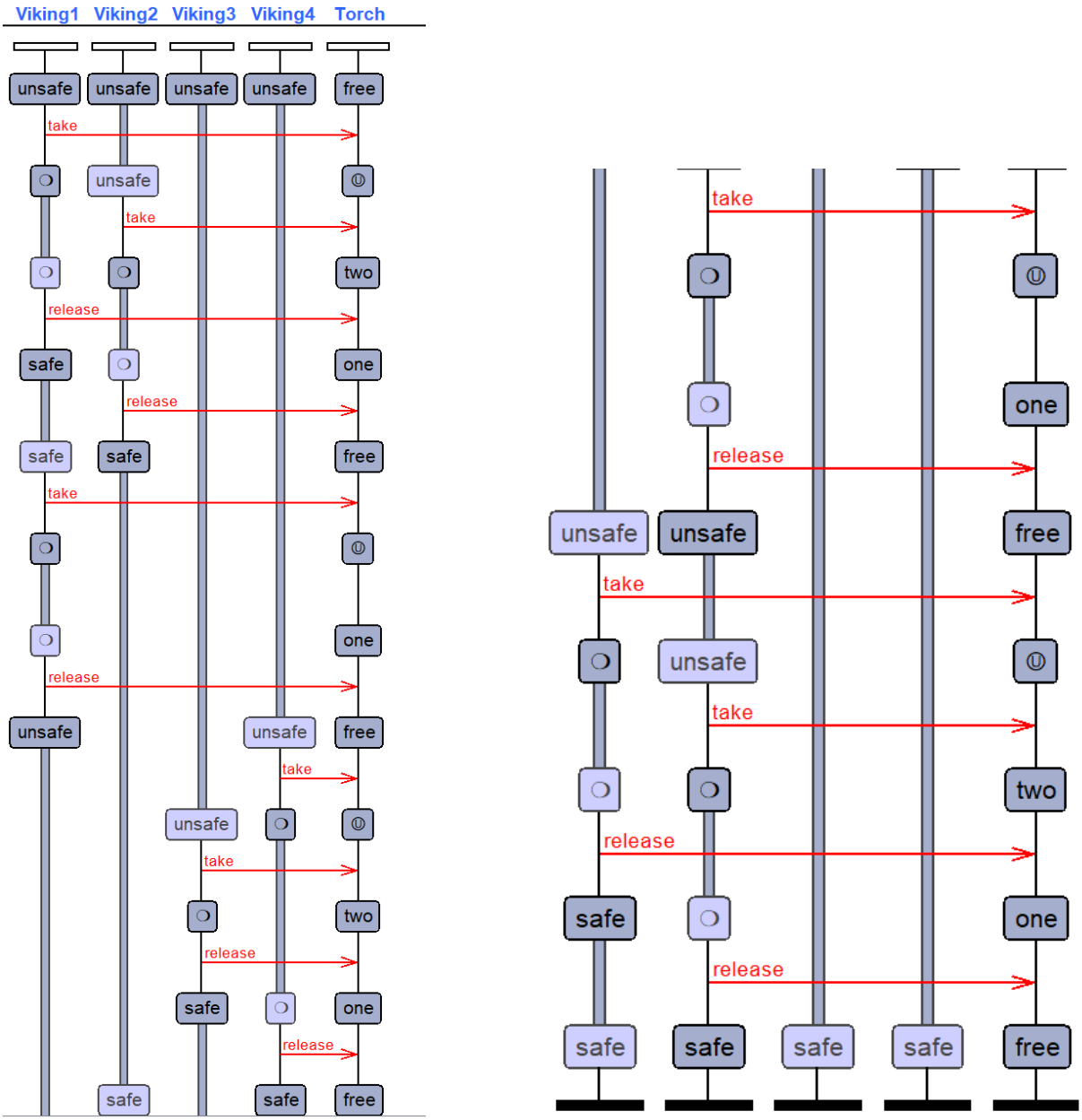
Witness schedule from trace:

#	Trip (Vikings by index; times in minutes)	Start time (min)	Trip origin
1	$\text{Vik}_1 + \text{Vik}_2$ cross to safe side (5+10)	0	Unsafe side
2	Vik_1 returns with torch (5)	10	Safe side
3	$\text{Vik}_3 + \text{Vik}_4$ cross to safe side (20+25)	15	Unsafe side
4	Vik_2 returns with torch (10)	40	Safe side
5	$\text{Vik}_1 + \text{Vik}_2$ cross to safe side (5+10)	50	Unsafe side

Table 1: A.1 crossing schedule (total completion time: 60 min).

The schedule completes all crossings at minute 60, satisfying the time bound exactly.

Figure 1: A.1: UPPAAL verifier for bounded reachability within 60 min



(a) First segment.

(b) Second segment.

Figure 2: A.1: UPPAAL verifier/trace views for bounded reachability within 60 min (two panels).

A.2: Add a fifth Viking (10 minutes)

A fifth Viking is added with a 10 min *bridge delay (in addition to the existing* 10 min *Viking). What is the shortest time for all five Vikings to reach the safe side? Describe model changes and queries used.*

The fifth Viking is added as a separate template instance with delay `slowest2=10`, composed with the existing four Vikings and the torch:

```
const int slowest2 = 10;
Viking5 = Soldier(slowest2);
system Viking1, Viking2, Viking3, Viking4, Viking5, Torch;
```

Core reachability query (bounded completion time T):

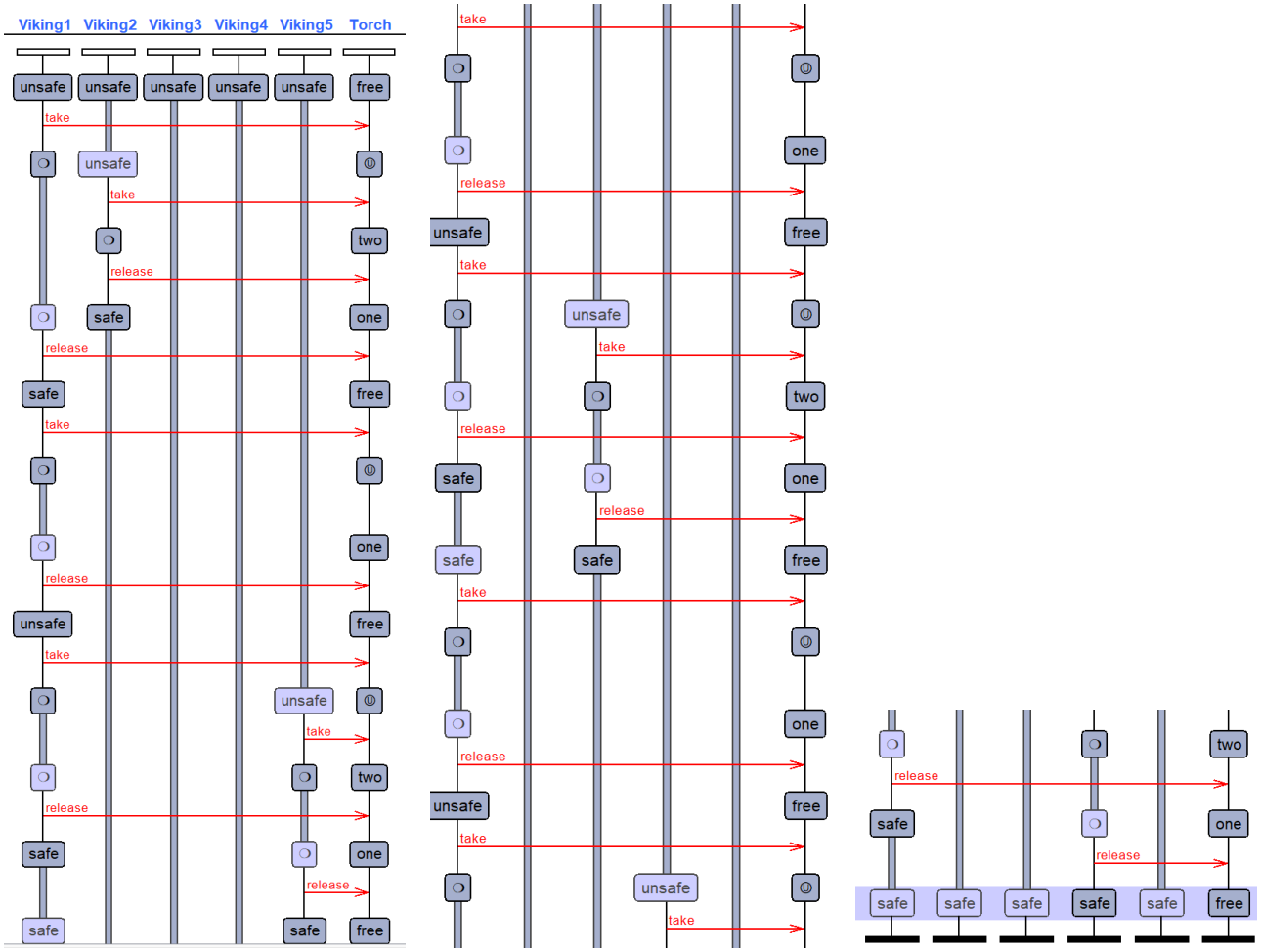
```
E<> Vik_1.safe && Vik_2.safe && Vik_3.safe && Vik_4.safe && Vik_5.safe && time <= T
```

A decreasing search over T and simulator tracing support the schedule below. In the optimal trace, Vik_1 and Vik_2 cross first and Vik_1 returns; then Vik_1 and Vik_5 cross and Vik_1 returns; next the slow pair Vik_3 and Vik_4

crosses together; Vik_2 returns with the torch; and the final trip is Vik_1 and Vik_2. The shortest completion time obtained for this model is 75 min.

#	Trip (Vikings by index; times in minutes)	Start time (min)	Trip origin
1	$\text{Vik}_1 + \text{Vik}_2$ cross to safe side (5+10)	0	Unsafe side
2	Vik_1 returns with torch (5)	10	Safe side
3	$\text{Vik}_1 + \text{Vik}_5$ cross to safe side (5+10)	15	Unsafe side
4	Vik_1 returns with torch (5)	25	Safe side
5	$\text{Vik}_3 + \text{Vik}_4$ cross to safe side (20+25)	30	Unsafe side
6	Vik_2 returns with torch (10)	55	Safe side
7	$\text{Vik}_1 + \text{Vik}_2$ cross to safe side (5+10)	65	Unsafe side

Table 2: A.2 crossing schedule (different ordering of Vikings also possible; total completion 75 min).



(a) First segment. (b) Second segment. (c) Third segment.

Figure 3: A.2: UPPAAL simulator trace split across three views.

A.3: Two-bridge scenario

Building on A.2, five Vikings may choose between two bridges: the original bridge (capacity 2, normal crossing delay) and a new bridge (capacity 3, each Viking takes 2× their original bridge delay). Both bridges require the same torch, so only one bridge can be in use at a time. Determine the shortest completion time and provide a schedule with a fourth column indicating original vs. new bridge.

We updated the model to match the official two-bridge statement. The specific UPPAAL changes are:

- The Torch template now makes a non-deterministic decision of selecting which bridge to use. This unifies

the delay penalties across all Vikings by setting a global bridge selector variable based on the chosen bridge.

- A new global declaration was added for the bridge selector, named `B`. We use `B==0` for Bridge 1 and `B==1` for Bridge 2.
- An urgent location was added in the Torch template so the choice between bridges occurs without time elapse at the decision point.

Reference screenshots of the modified templates used for A.3 are shown in Fig. 4 and Fig. 5.

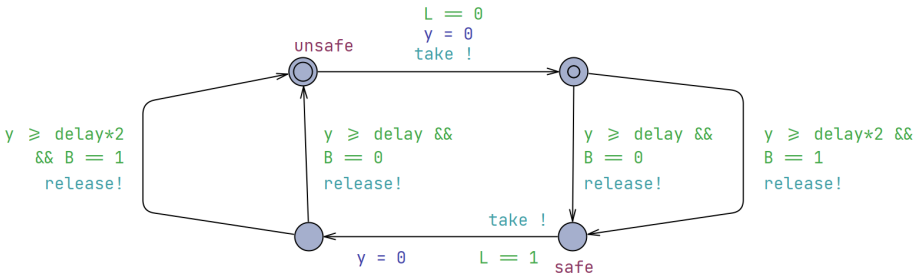


Figure 4: A.3: Modified Viking (Soldier) template used in the two-bridge model.

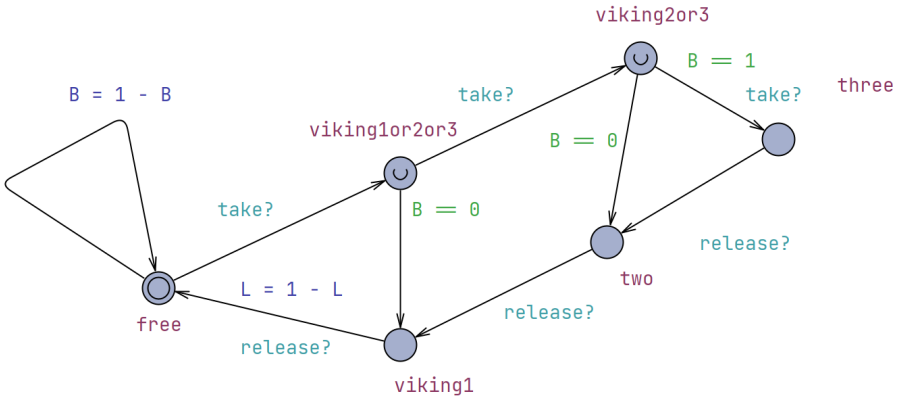


Figure 5: A.3: Modified Torch template with an urgent bridge-selection choice and bridge selector `B`.

The correct bounded reachability query for A.3 is:

```
E<> Viking1.safe && Viking2.safe && Viking3.safe && Viking4.safe && Viking5.safe && time <= 70
```

We determined satisfiability by incrementally changing the time bound. The shortest completion time observed for this model is 70 min, and a witness trace exists for the query above.

From the trace, the execution first uses Bridge 1 (`B==0`) for the two slow-pair structure to move Vik_3 and Vik_4, then finishes by using Bridge 2 (`B==1`) to carry three Vikings together with the required delay penalty. The total time in the trace is 70 min.

#	Trip (Vikings by index; times in minutes)	Start time (min)	Trip origin	Bridge used
1	$\mathrm{Vik}_1 + \mathrm{Vik}_2$ cross to safe side (5+10)	0	Unsafe side	Bridge 1 (`B==0`)
2	Vik_1 returns with torch (5)	10	Safe side	Bridge 1 (`B==0`)
3	$\mathrm{Vik}_3 + \mathrm{Vik}_4$ cross to safe side (20+25)	15	Unsafe side	Bridge 1 (`B==0`)
4	Vik_2 returns with torch (10)	40	Safe side	Bridge 1 (`B==0`)
5	$\mathrm{Vik}_1 + \mathrm{Vik}_2 + \mathrm{Vik}_5$ cross to safe side (2×10)	50	Unsafe side	Bridge 2 (`B==1`)

Table 3: A.3 witness schedule reaching completion at 70 min. (Bridge selection shown via `B` in the satisfying trace.)

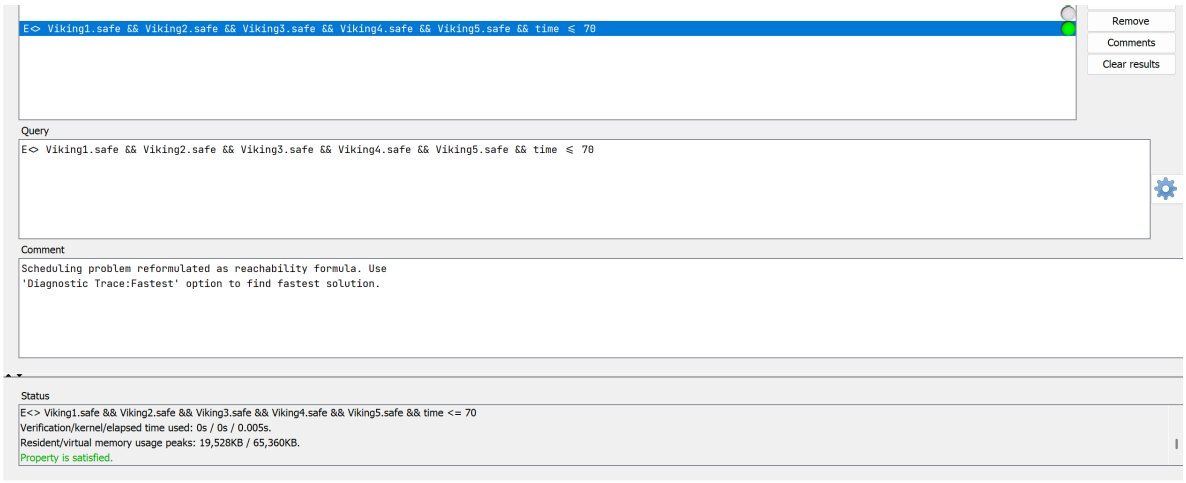


Figure 6: A.3: Verifier screenshot showing the bounded reachability query is satisfied for `time <= 70`.

2 Part B: Traffic-Light and Crosswalk Verification

B.1: Discrete-model reasoning (before timed implementation)

For each property, a reachability-oriented check was defined conceptually and then evaluated by model intuition over the discrete behavior (no tool reachability on the discrete model in this step).

Prop.	Reachability check idea	Expected truth	Rationale (intuition)
(a)	It is impossible for the traffic light to be in the pending state 70 s after the initial condition.	False	Reachable if the pedestrian button is pressed shortly after entering green, so pending can still be active at 70 s.
(b)	It is possible for the traffic and crosswalk lights to be green at the same time.	False	The pedestrian side exits green at around 55 s, while the traffic side needs at least 60 s before returning to green, so overlap is excluded in this discrete intuition.
(c)	The traffic light will never stay in the green (non-pending) location longer than 120 s.	True	The location invariant forces a jump by 120 s; the controller cannot remain green beyond that bound.
(d)	The traffic light will never stay in the pending location for 45 s or longer.	False	If `pedButton` is pressed immediately at green entry, pending can persist beyond 45 s.
(e)	The traffic light will never stay in the pending location for 65 s or longer.	True	Pending can extend to about 60 s in worst case, but not to 65 s.
(f)	It is possible that the crosswalk is red while the traffic light is green.	True	This is a normal reachable operating phase of the controller pair.
(g)	Whenever the crosswalk is red, the traffic light must be green.	False	Counterexample: yellow traffic with red crosswalk is reachable.
(h)	The 3rd occurrence of `sigG` must not happen within the first 300 s after the initial condition.	True	With approximate period 125 s (without extra pedestrian stall), only two green events should occur before 300 s.
(i)	The 3rd occurrence of `sigG` must not happen within the first 400 s after the initial condition.	False	By around 375 s, repeated cycles allow multiple green events; the statement does not hold at 400 s.

Table 4: B.1 qualitative checks and expected outcomes (intuition only).

B.2: Timed automata model construction

The timed UPPAAL model includes:

- Traffic-light and crosswalk-light automata connected by broadcast/synchronization channels (e.g., `sigR`, `sigG`, `sigY`, `pedG`, `pedR`).
- Non-deterministic pedestrian input trigger (`pedinput`): implemented as a self-looping transmitter so that the pedestrian button channel can fire concurrently with other synchronizations. The self-loop does not encode meaningful defaulting behavior; it exists so that UPPAAL's channel semantics allow the input to be offered in the same step as other transitions when needed.
- Receiver/sink automata for outputs that would otherwise lack a listener: unused outputs are connected to simple self-looping receivers that absorb signals, which keeps broadcast sends from blocking the rest of the model.
- Urgent intermediate locations where needed to split transitions that conceptually combine input and output events.

The model was exercised with concrete simulation and simple reachability checks before formal verification in B.3.

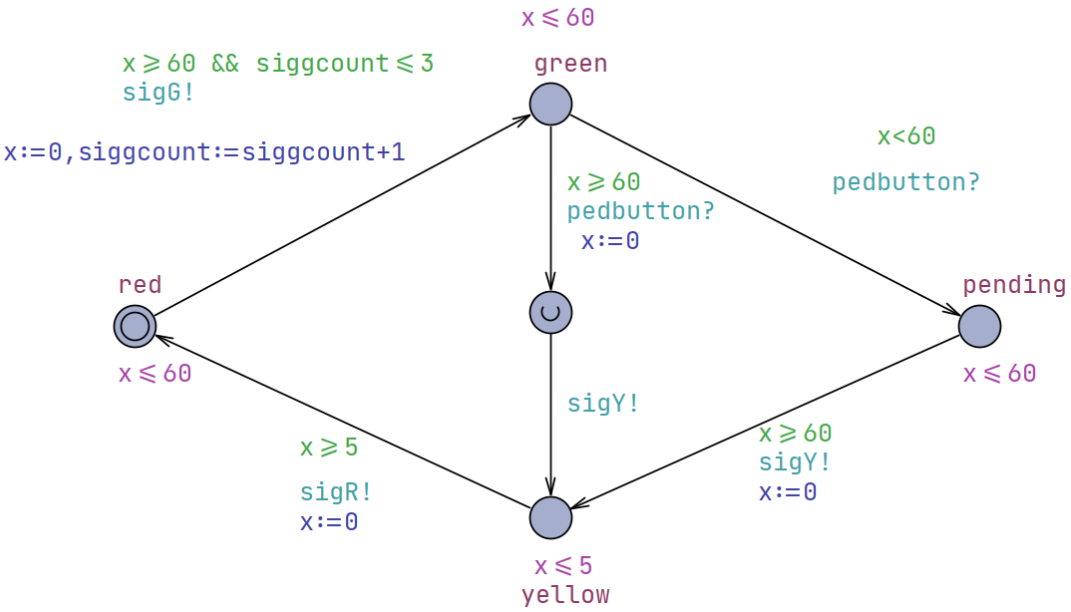


Figure 7: Timed automata for the traffic-light system (B.2).

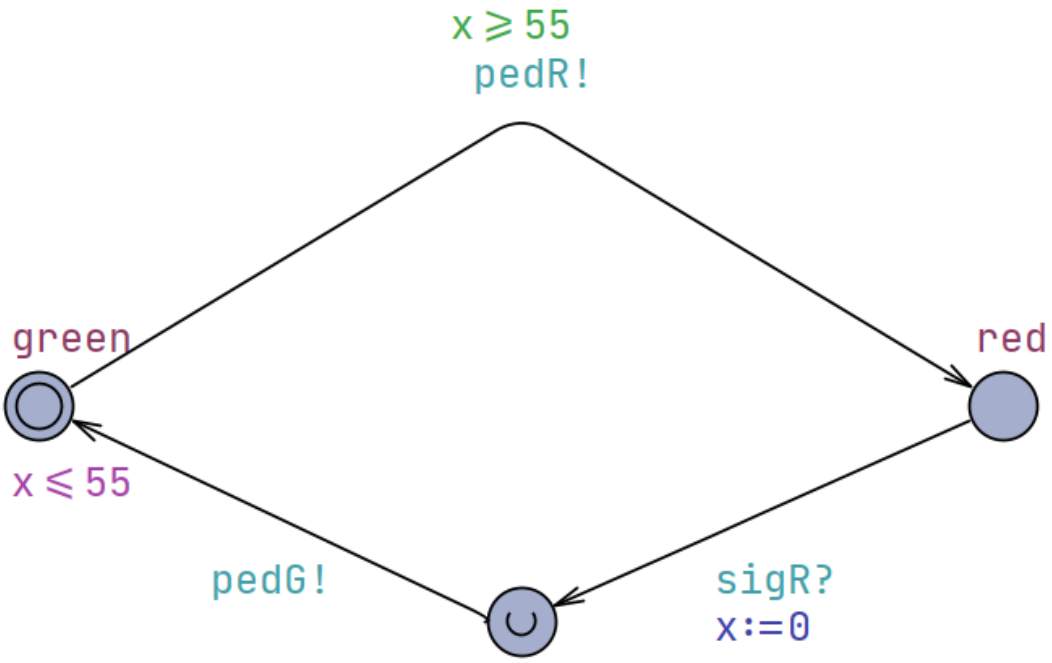


Figure 8: Crosswalk / pedestrian-side signal system (B.2).

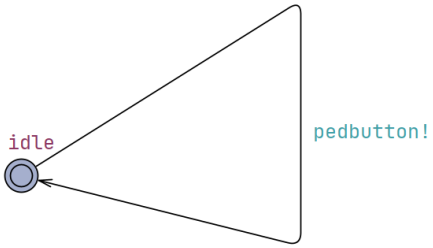


Figure 9: Non-deterministic `pedinput` trigger (self-loop for channel firing). Image width is half of the first two B.2 figures.

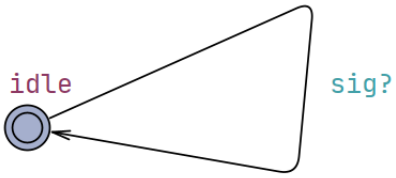


Figure 10: Receiver/sink automata for unused outputs. Image width is half of the first two B.2 figures.

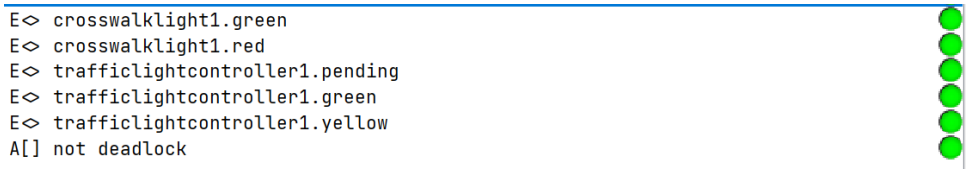


Figure 11: Reachability / sanity checks used while testing the B.2 model.

B.3: Query mapping and verification outcomes

Each B.1 English statement was translated into a UPPAAL query over the B.2 timed model. The verifier panel confirms results for all checks in one model snapshot.

Prop.	Representative UPPAAL query form	UPPAAL outcome	Final statement truth
(a)	`E<> (globalTime==70 && traffic.pending)`	Reachable	False
(b)	`E<> (traffic.green && crosswalk.green)`	Reachable (timed model)	False
(c)	`A[] (traffic.green imply tGreen<120)` and `E<> (traffic.green && tGreen>=120)`	Satisfied / Not reachable	True
(d)	`A[] not (traffic.pending && tPend>=45)`	Violated	False
(e)	`A[] not (traffic.pending && tPend>=65)`	Satisfied	True
(f)	`E<> (traffic.green && crosswalk.red)`	Reachable	True
(g)	`A[] (crosswalk.red imply traffic.green)`	Violated	False
(h)	`A[] (sigGCount<3 or globalTime>300)`	Satisfied (target interpretation)	True
(i)	`A[] (sigGCount<3 or globalTime>400)`	Violated	False

Table 5: B.3 query mapping with verifier outcomes and final truth values for the original English statements.

Note. Some formulas are written as reachability checks of counterexamples. Therefore, a query outcome and the final truth of the original English statement can be opposite by design.

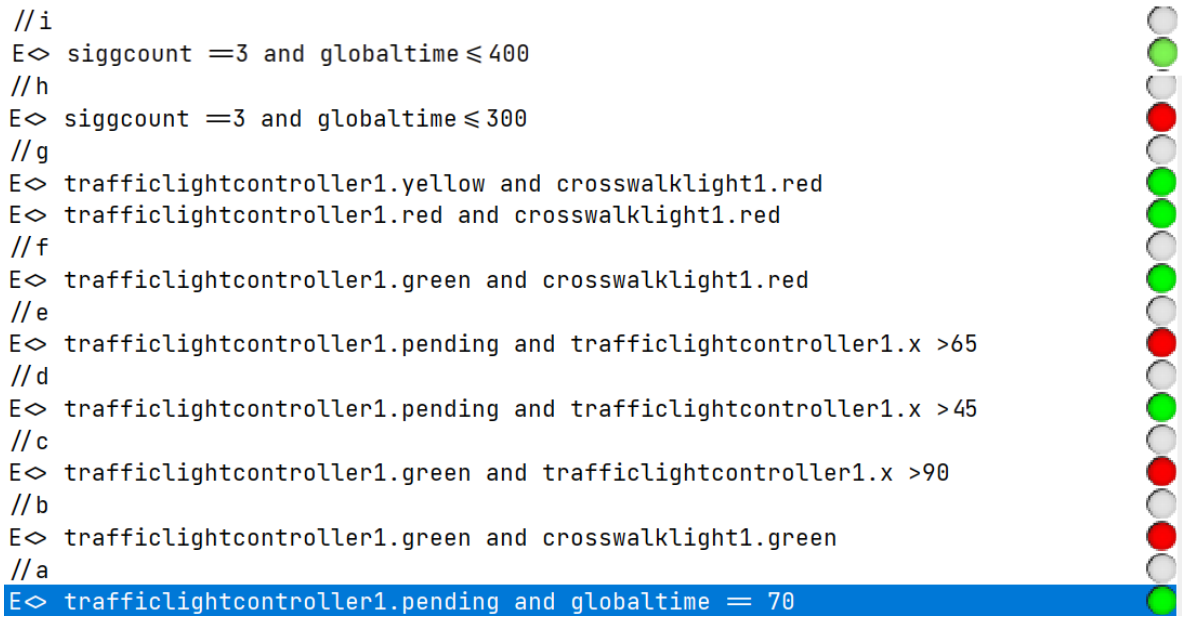


Figure 12: Verifier window: B.3 queries with pass/fail indicators.

B.4: Pedestrian component and unsafe crossing-time range

The previous button-generator and `pedG` receiver were replaced with an explicit pedestrian automaton having three locations: *Idle*, *Waiting*, and *Crossing*. Parameter `pedCrossTime` controls required crossing time.

Bad condition: the system is unsafe when traffic becomes green while the pedestrian remains in the crossing location.

Observed threshold: reachability checks show the bad condition is unreachable at `pedCrossTime=59` but reachable at `pedCrossTime=60`. The smallest unsafe value is therefore 60, and values ≥ 60 are unsafe for this model.

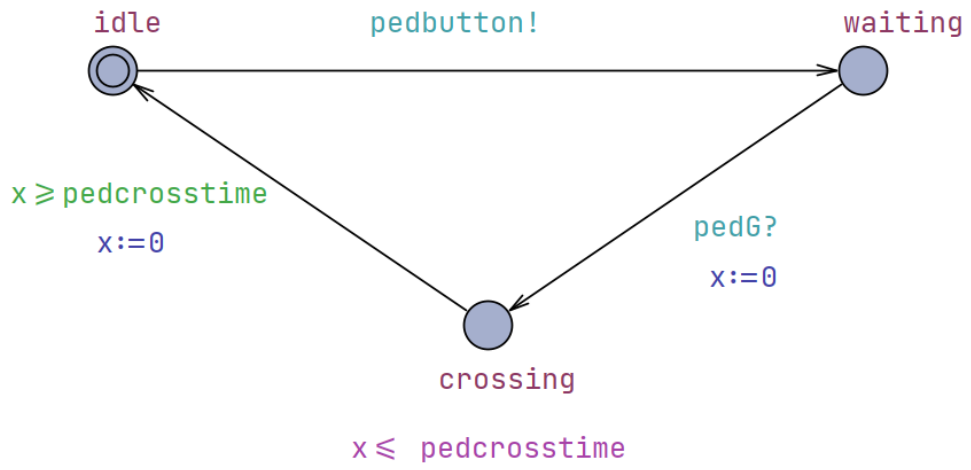


Figure 13: Pedestrian template with `pedCrossTime` and crossing-state timing (B.4).

Figure 14: B.4 safe case at `pedCrossTime=59`: query/clause (top) and trace (bottom).

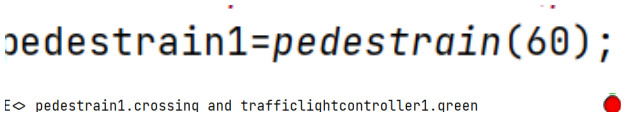


Figure 15: B.4 unsafe case at `pedCrossTime=60`: query/clause (top) and trace (bottom).

B.5 (ECE 622): Robustness change for arbitrary crossing time

Lab 2 asks for a more robust model such that safety holds regardless of how long the pedestrian takes to cross; explain the change; repeat the B.4 safety query at a `pedCrossTime` value that was unsafe before and show it is safe after the change; submit `_b5.xml` with the report.

To remove dependence on a bounded pedestrian crossing duration, a synchronization signal `done` was introduced from the pedestrian automaton to the traffic controller. The controller defers returning to green until the pedestrian reports crossing completion.

The previously unsafe test point from B.4 (`pedCrossTime=60`) becomes safe after adding the `done` handshake and updating controller guards accordingly.

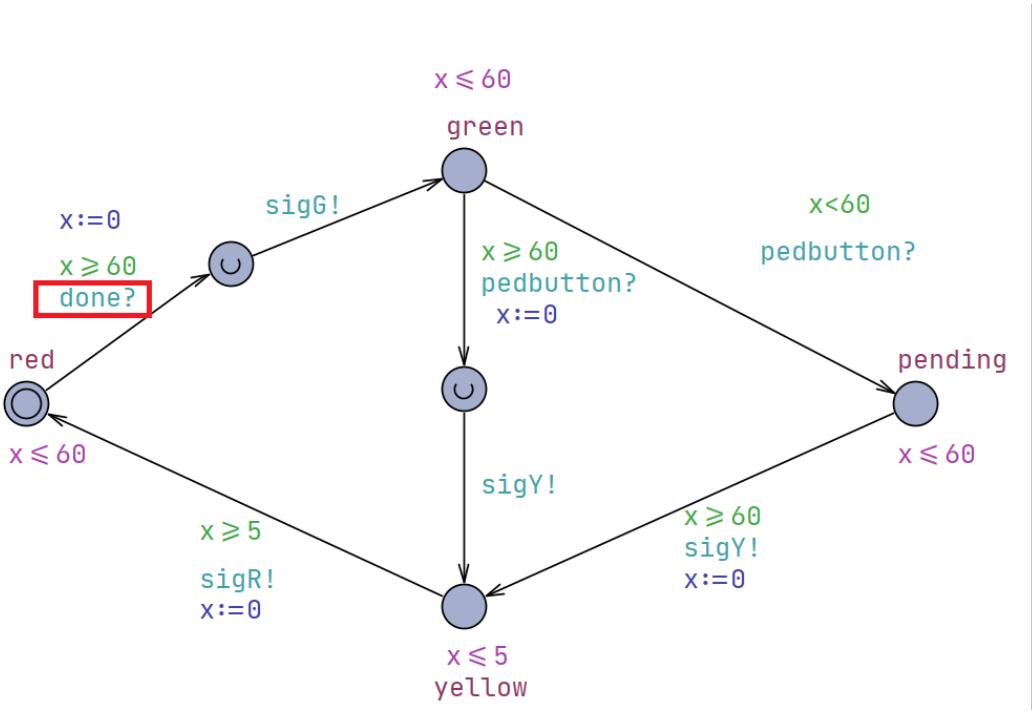


Figure 16: Traffic controller with `done` handshake (B.5).

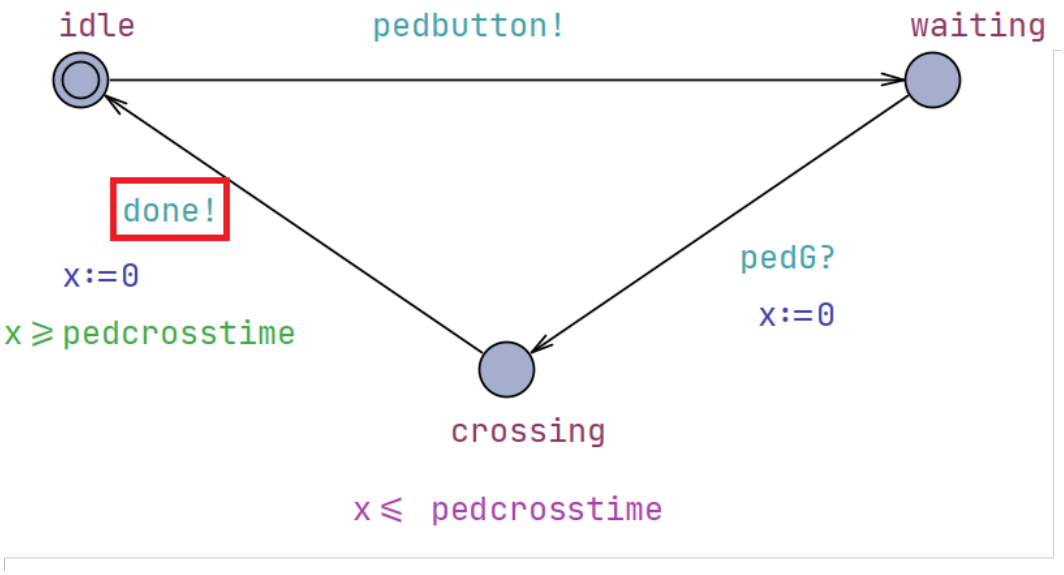


Figure 17: Pedestrian model cooperating with `done` (B.5).

Figure 18: B.5: safety re-check at `pedCrossTime=60` (now safe).

```
pedestrain1=pedestrain(60);
```

Figure 19: B.5: corresponding declaration line for the unsafe condition at `pedCrossTime=60`.

Conclusion

This lab helped us understand how timing constraints and synchronization mechanisms like channels, can be used to model and verify complex systems. We also learned how to use UPPAAL to model and verify systems, and how to use the tool to extract schedules and traces. Part A.3 Torch modeling was a good exercise to finally get it right! Thank you for a great lab!

Appendix: Tooling and Submission Artifacts

This report was prepared using Cursor AI and compiled with `latexmk` on the VLSICAD server, using the lab project working directory as the build root.

Cursor was also used to iteratively edit and format the LaTeX source (tables, figures, and UPPAAL query listings) while keeping the report consistent with the lab writeup. However, heavily critical verification was done by us to make tiny details are consistent with the lab writeup. Due to hallucinations, sometimes reiterations or clarifications were needed to make AI understand the context of problems and difference between true statements and opinions/hypotheses.

For modeling and simulation, UPPAAL was run on a Windows PC; screenshots/snapshots generated during UPPAAL runs were committed alongside the LaTeX sources and synchronized via GitHub, so the Linux build environment and the Windows modeling environment stayed aligned on the same report inputs.

Final submission model files

- `shih_a2.xml`
- `shih_a3.xml`
- `shih_b3.xml`

- shih_b4.xml
- shih_b5.xml

Repository link

https://github.com/JDKhare/UMass_ECE654_Projects